

§ Lire et écrire des données persistantes §

PRÉREQUIS

Avant de pouvoir commencer la séance, vous devez :

1. avoir terminé les exercices précédents ;
2. avoir lu les slides de la séance du jour.

OBJECTIFS DE LA SÉANCE

1. Créer un service permettant de lire et d'écrire des données persistantes.
2. Créer des services permettant de gérer les ressources.
3. Modifier les contrôleurs pour utiliser ces services.

INTRODUCTION

Avant tout chose, prenez les fichiers présents sur MooVin :

- `files.service.ts` : placez ce fichier dans le dossier `src/services/`,
- `guards.ts` : placez ce fichier dans le dossier `src/utils/` et remplacez le fichier actuel.

À la racine de votre projet (au même niveau que `src`), créez un dossier `data` qui contiendra nos données. C'est dans ce dossier que se trouveront les différents fichiers JSON utilisés pour stocker les données de façon persistante.

Lisez attentivement le service `files.service.ts`. Ce service permet de lire et d'écrire des données sous format JSON dans un fichier. Faites en particulier attention au caractère générique des fonctions. En cas de besoin, revoyez les slides liées à cette séance.

EXERCICE 1: UTILISATION DU FILESERVICE

CRÉATION DES MODÈLES

Créez et exportez une interface `DoctorDBO` (DBO pour Database Object) au sein du fichier `doctor.model.ts`. Cette interface représentera la façon dont les docteurs seront stockés dans le fichier JSON. Cette interface a les propriétés suivantes :

- `id` : un nombre,
- `first_name` : une chaîne de caractères,
- `last_name` : une chaîne de caractères,
- `speciality` : une chaîne de caractères.

Notez la notation `snake_case` des champs enregistrés dans le fichier JSON.

MODIFICATION DU MAPPER

Créez deux nouvelles fonctions `toDBO` et `fromDBO` au sein du fichier `doctors.mapper.ts`. Ces fonctions ont comme spécifications :

- `toDBO` : convertit un `Doctor` en `DoctorDBO`,
- `fromDBO` : convertit un `DoctorDBO` en `Doctor`.

Vous enregistrez et lisez donc bien une ressource `DoctorDBO` dans vos fichiers, vos données doivent donc être enregistrées dans un fichier JSON avec les noms de champs type `first_name` et non `firstName`.

Créez un fichier `data/doctors.json` et insérez-y un docteur de façon à pouvoir retourner un docteur. Choisissez des valeurs pour les champs `first_name`, `last_name` et `speciality`. Donnez à ce docteur l'id 1. **Attention** : tous les fichiers JSON doivent contenir une **table**, par exemple :

```
1 [
2   {
3     "id": 1,
4     "first_name": "john",
5     "last_name": "doe",
6     "speciality": "cardiologist"
7   }
8 ]
```

EXERCICE 1: CRÉATION DU DOCTORSERVICE

En vous référant aux slides, créez un service `services/doctors.service.ts`. Ce service utilisera le `FilesService` pour lire et écrire des données persistantes.

Codez la fonction `getAll` du service `DoctorsService`. Cette fonction a comme spécification :

- `getAll` : retourne un tableau de docteurs,

Cette fonction doit :

- Utiliser le filtre `DoctorFilter` fourni en entrée pour filtrer les docteurs.
- Lire le fichier `data/doctors.json` en utilisant le service `FilesService`. Attention : cette fonction doit être utilisée dans un bloc try-catch pour gérer les erreurs. En cas d'erreur, celle-ci doit être loggée et la fonction renvoie une table vide.

- Convertir les données lues (DoctorDBO) en `Doctor` en utilisant la fonction `fromDBO` du mapper.

Modifiez votre contrôleur `DoctorsController`, en important votre service et en l'utilisant pour répondre à la requête HTTP GET <http://localhost:3000/doctors>.

A l'aide du fichier `http/doctors.http`, testez que votre serveur fonctionne correctement et retourne bien la liste des docteurs que vous avez inséré(s) dans le fichier.

EXERCICE 2: FINALISEZ LE SERVICE DOCTOR

Complétez le service `doctors.service.ts` avec les fonctions suivantes :

- `getById` : retourne un docteur en fonction de son `id`. Attention, cette fonction retourne `undefined` si le docteur n'existe pas.
- `create` : insère un docteur dans le fichier `data/doctors.json`, retourne le docteur inséré ou `undefined` si le `FilesService` échoue à lire ou à écrire le fichier.
Attention : Cette fonction attribue l'id du nouveau docteur. De façon à être tout à fait cohérent avec le reste de l'application, l'id du nouveau docteur doit être le plus grand id actuel + 1. Il faudra donc calculer le `max` de tous les ids.
- `update` : met à jour un docteur dans le fichier `data/doctors.json`, retourne le docteur mis à jour ou `undefined` si le docteur n'existe pas ou si le `FilesService` échoue à lire ou à écrire le fichier ;
- `delete` : supprime un docteur du fichier `data/doctors.json` et renvoie un booléen indiquant si la suppression a réussi ou non.

Notez bien le nom de ces fonctions. Ils sont **essentiels** ! En effet, les tests unitaires que vous allez réaliser par la suite se basent sur ces noms.

De plus, ces noms seront utilisés dans tous nos services pour les opérations de base. Ils constituent l'API de bases des services.

Refactoring n'hésitez pas à retravailler votre service. En particulier veillez à **minimiser la duplication de code**. Créez d'autres fonctions et variables statiques que celles listées ci-dessus si nécessaire. Un bon code n'est pas seulement un code qui fonctionne, mais aussi un code qui est facile à maintenir et à comprendre.

EXERCICE 3: LE MODÈLE PATIENTS

Faites les mêmes opérations pour la ressource `Patient`.

LE DATA BASE OBJECT DU PATIENT

Créez une interface `AddressDBO` dans le fichier `models/address.model.ts` contenant les champs :

- `street` : une chaîne de caractères,
- `number` : une chaîne de caractères,
- `city` : une chaîne de caractères,
- `zip_code` : une chaîne de caractères,
- `country` : une chaîne de caractères.

Créez une interface `PatientDBO` au sein du fichier `models/patient.model.ts`. Cette interface représentera la façon dont les patients seront stockés dans le fichier JSON. Cette interface a les propriétés suivantes :

- `id` : un entier servant de clé primaire,
- `first_name` : une chaîne de caractères,
- `last_name` : une chaîne de caractères,
- `birthdate` : une chaîne de caractères,
- `niss` : une chaîne de caractères,
- `ref_doctor` : un entier faisant référence à un docteur,
- `address` : `AddressDBO`.

Tous les champs sont obligatoires. Mettez à jour le fichier `/data/patients.json` et insérez un patient à votre base de données. Par exemple,

```
1 [{
2   "id": 2,
3   "first_name": "Averoes",
4   "last_name": "Ibn Rochd",
5   "birthdate": "1926-05-21",
6   "niss": "260511-123-45",
7   "ref_doctor": 1,
8   "address": {
9     "number": 2,
10    "street": "rue de la poesie",
11    "city": "Uccle",
12    "country": "Belgique",
13    "zip_code": "1200"
14  }
15 }]
```

MODIFICATION DU MAPPER PATIENTSMAPPER

Implémentez les fonctions permettant de convertir un `Patient` en `PatientDBO` et vice versa.

Attention : les dates doivent être gérées correctement. Lorsque vous créez un objet de type `PatientDBO`, vous devez convertir la date de naissance en une chaîne de caractères en utilisant la méthode `toISOString()` de l'objet `Date`. Inversement, lorsque vous créez un objet `Patient` créez une nouvelle date : `new Date(dbo.birthdate)`.

LE SERVICE PATIENTSSERVICE

Créez un fichier `services/patients.service.ts` et implémentez les fonctions permettant de gérer le CRUD des patients.

En particulier, la fonction `create` doit avoir le comportement suivant :

- elle vérifie qu'il n'existe pas de patient avec le même `niss`,
- si le patient existe, elle log une erreur et retourne `undefined`,
- la fonction utilise le `DoctorsService` pour vérifier que le médecin de référence existe,
- si le médecin n'existe pas, elle log une erreur et retourne `undefined`,

Importez ce service dans votre contrôleur `PatientsController` et utilisez-le pour répondre aux requêtes HTTP. Vérifiez que votre serveur fonctionne correctement en testant les différentes routes.

Reprenez bien les noms présentés dans la première partie de la fiche pour les fonctions de base du service de la ressource `Patient`.

Commencez par le CRUD de base du patient sans tenir compte du filtrage des patients. Une fois le CRUD terminé et testé, vous pourrez passer à la suite.

Nommez les fonctions de manière camelCase et donnez des noms les plus explicites possibles. Les fonctions doivent être des verbes d'action. Par exemple, comme les fonctions restantes permettent d'obtenir des données, elles commencent par `get`, elles permettent d'obtenir des données en fonction d'une propriété, elles commencent par `getBy`. Ensuite, c'est le nom de la propriété qui est utilisé. Par exemple, pour obtenir un patient en fonction de son `zipCode`, la fonction s'appellera `getByZipCode`.

EXERCICE 4: AJUSTEZ VOS FILTRAGES

Les filtres implémentés sur les routes `GET /patients` et `GET /doctors` doivent continuer à être fonctionnels. Ce sont bien les services qui filtrent les données, pas les contrôleurs. Les fonctions `getAll` doivent donc pouvoir prendre un paramètre `filter` de type `PatientFilter` ou `DoctorFilter`.